

Relatório técnico de refatoração Spark e Airflow

Separação de responsabilidades de execução na camada Silver e Data Quality

Projeto de TCC — Engenharia de Dados

9 de setembro de 2026

Sumário

1	Objetivo e resultado	1
2	Contexto técnico e escopo	2
3	Auditoria inicial	2
3.1	Classificação adotada	2
3.2	Configurações encontradas na Silver	2
4	Implementação realizada	3
4.1	Organização das responsabilidades	3
4.2	Configurações por task	3
4.3	Dependência compartilhada com Gold	4
4.4	Inventário dos arquivos	4
5	Preservação do fluxo e das falhas	5
6	Validações e resultados	5
7	Auditoria Gold: pendências preservadas	6
8	Estatísticas e revisão do diff	6
9	Limitações e próximos passos	7
10	Base documental	7

1 Objetivo e resultado

Este relatório documenta a auditoria e a refatoração dos jobs PySpark já orquestrados pela DAG `dags/tcc_pipeline.py`. O objetivo foi retirar dos scripts de processamento as configurações fixas de infraestrutura e execução, transferindo para o Airflow as configurações necessárias e preservando os valores específicos de tuning.

A implementação foi concluída e validada por verificações de sintaxe, imports, estrutura da DAG e simulações de falha. A DAG continua com quatro TaskGroups e dez tasks. Os scripts de

transformação e de Data Quality efetivamente utilizados por ela não definem mais master, interface Spark, interpretadores Python ou configurações de shuffle nos builders de sessão.

Não houve execução completa dos datasets após a refatoração. Portanto, este documento distingue a preservação do código funcional, verificada no diff, da equivalência dos dados produzidos, que não foi medida por reproprocessamento.

2 Contexto técnico e escopo

O projeto utiliza Docker, Airflow e Spark/PySpark. Os arquivos de ambiente consultados especificam Airflow 3.3.1, provider Apache Spark 6.3.2, PySpark 3.5.0, Python 3.11 e Java 17. A DAG utiliza `SparkSubmitOperator` e referencia a connection `spark_default`.

Segundo o contexto fornecido pelo responsável pelo projeto, essa connection controla master `local[2]`, deploy mode `client` e binário `spark-submit`. A refatoração preservou a referência à connection e não inseriu `spark.master` no `conf`. Os valores armazenados na connection não foram consultados diretamente durante as validações registradas.

O escopo de implementação compreendeu cinco scripts Silver, cinco scripts DQ Silver, um novo helper de sessão e a DAG existente. Não foram alterados Bronze, Gold, regras de qualidade, schemas, joins, caminhos de entrada e saída, formatos Parquet, modos de escrita, particionamento físico explícito ou valores esperados nas validações.

Docker Compose, Dockerfiles e requirements permaneceram intactos. O container de desenvolvimento Spark foi preservado. Não foram executados comandos destrutivos, mudanças de permissões, commit ou push.

3 Auditoria inicial

Foram pesquisadas ocorrências de `SparkSession.builder`, `.master(...)`, configurações `spark.*`, caminhos `/opt/conda`, variáveis Python e configurações de recursos. A inspeção cobriu código, DAG, ambiente Docker, testes e referências textuais do repositório.

3.1 Classificação adotada

- **Infraestrutura/runtime:** master, interface Spark e seleção dos interpretadores Python. Identificam onde e como o processo é executado.
- **Tuning/performance:** número de partições de shuffle, Arrow e limite de broadcast automático. Influenciam execução, conversão e planejamento; seus valores existentes exigem preservação deliberada.
- **Comportamento necessário ao processamento:** schemas, opções de leitura, transformações, regras de qualidade e escrita. Não foram removidos apenas por pertencerem à API Spark.

`appName` foi preservado como identificação do job. O nível de log `ERROR` também foi mantido, conforme permitido no escopo. `SparkSession.builder` é um mecanismo de criação da sessão, não um hardcode de infraestrutura por si só.

3.2 Configurações encontradas na Silver

Origem	Configurações anteriores
INMET horário	Master local[2]; shuffle 4; UI false; Python do driver e workers em /opt/conda/bin/python; Arrow false.
INMET diário	Shuffle 40; UI false.
IBGE	Shuffle 4; UI false.
CID10	Master local[2]; UI false.
SIH	Master local[2], na função criar_spark de sih.py.
Helper DQ compartilhado	Master local[2]; UI false; shuffle 4.

Não foram encontrados hardcodes de memória do driver/executor ou executor cores nesses builders. As referências /opt/conda em documentação e saídas de notebooks não pertencem à execução dos jobs da DAG e permaneceram fora da refatoração. O smoke test existente também mantém sua configuração própria de UI, fora do escopo dos scripts orquestrados.

4 Implementação realizada

4.1 Organização das responsabilidades

Os scripts continuam responsáveis pela transformação e validação. A DAG passou a concentrar as configurações de execução e tuning removidas. A imagem Airflow continua fornecendo dependências e ambiente. A connection permanece responsável pelo master.

O padrão de criação de sessão adotado foi:

```
spark = (
    SparkSession.builder
        .appName("nome-do-job")
        .getOrCreate()
)
```

4.2 Configurações por task

SPARK_CONF mantém spark.ui.enabled=false, já presente na DAG antes da mudança. Foram acrescentados três dicionários que reutilizam essa base:

- SPARK_CONF_SHUFFLE_4: preserva shuffle 4 para IBGE e todos os DQs Silver.
- SPARK_CONF_INMET: reutiliza shuffle 4, preserva Arrow false e seleciona python3 para driver e workers.
- SPARK_CONF_INMET_DIARIO: preserva shuffle 40.

Task	Shuffle configurado	Configurações adicionais
silver_inmet	4	Arrow false; ambos os interpretadores python3.
silver_inmet_diario	40	Base comum.
silver_ibge	4	Base comum.
silver_cid10	Não imposto	Base comum.
silver_sih	Não imposto	Base comum.
Cinco tasks dq_silver_*	4	Base comum.

Todas as tasks mantêm UI false. Não impor shuffle a CID10 e SIH preserva a ausência dessa configuração explícita nos scripts anteriores; não significa que Spark deixe de possuir um valor efetivo padrão.

As propriedades `spark.pyspark.python` e `spark.pyspark.driver.python` do INMET foram transferidas para a DAG com valor `python3`. O worker consultado resolve esse comando para `/home/airflow/.local/bin/python3`. Assim, o código de transformação deixa de depender do caminho de outra imagem. O funcionamento de workers Spark com esse interpretador não foi exercitado por uma ação Spark nesta validação.

4.3 Dependência compartilhada com Gold

A auditoria revelou que `src/quality/gold/common.py` reexporta a função `criar_spark` de `src/quality/silver/common.py`. Remover as configurações desse helper alteraria indiretamente a execução do DQ Gold, apesar de não editar arquivos Gold.

Para preservar esse comportamento, o helper original foi mantido sem alterações. Criou-se `src/quality/silver/spark_session.py`, com a criação mínima de sessão e nível de log ERROR. Os cinco DQs Silver tiveram apenas seu import redirecionado para o novo módulo.

Essa decisão deixa uma dependência histórica a tratar em uma futura refatoração Gold. Uma busca em toda a pasta Silver ainda encontra hardcodes no antigo `common.py`, mas os DQs da DAG atual não o utilizam mais. A verificação de identidade das funções confirmou que o Gold continua usando a função original.

4.4 Inventário dos arquivos

Arquivo	Alteração e justificativa
<code>src/silver/inmet.py</code>	Remoção de seis chamadas de configuração/master do builder; centralização na execução Airflow.
<code>src/silver/inmet_diario.py</code>	Remoção de shuffle e UI; valores preservados na DAG.
<code>src/silver/ibge.py</code>	Remoção de shuffle e UI; valores preservados na DAG.
<code>src/silver/cid10.py</code>	Remoção de master e UI; responsabilidades externas.
<code>src/silver/sih.py</code>	Remoção de master e ajuste da descrição da sessão.
<code>src/quality/silver/dq_inmet.py</code>	Troca do import do helper.
<code>src/quality/silver/dq_inmet_diario.py</code>	Troca do import do helper.
<code>src/quality/silver/dq_ibge.py</code>	Troca do import do helper.
<code>src/quality/silver/dq_cid10.py</code>	Troca do import do helper.
<code>src/quality/silver/dq_sih.py</code>	Troca do import do helper.
<code>src/quality/silver/spark_session.py</code>	Novo helper sem hardcodes de execução.
<code>dags/tcc_pipeline.py</code>	Dicionários de configuração e associação por task.

No INMET, foram removidas cinco chamadas `.config(...)` — shuffle, UI, dois interpretadores e Arrow — e uma chamada `.master(...)`, totalizando seis chamadas. O conteúdo removido ocupa 15 linhas no diff.

5 Preservação do fluxo e das falhas

As dependências permanecem:

```
INMET: silver_inmet -> dq_silver_inmet
      -> silver_inmet_diario -> dq_silver_inmet_diario
IBGE:  silver_ibge -> dq_silver_ibge
CID10: silver_cid10 -> dq_silver_cid10
SIH:   silver_sih -> dq_silver_sih
```

As transformações mantêm `retries=1`, e os DQs, `retries=0`. Não foram adicionadas tasks Gold nem novas DAGs.

`processar_sih_lote.py` não foi alterado. A função de períodos continua retornando 36 competências, de janeiro de 2023 a dezembro de 2025. Uma simulação fez a primeira competência falhar e confirmou que as 36 foram tentadas, com `RuntimeError` ao final. Isso preserva o mecanismo de propagação necessário para o Airflow reconhecer falha; não foi disparada uma task real para observar seu estado na interface.

Os cinco gates DQ foram testados com relatório contendo erro crítico e sessão simulada. Cada `main()` propagou `ValueError` e chamou `spark.stop()`. As regras reais de validação e os dados não foram executados nessa simulação.

6 Validações e resultados

As verificações ocorreram no worker Airflow disponível, sem reprocessar dados. Foram usados `docker compose -f docker-compose.airflow.yaml exec -T airflow-worker`, comandos Python e consultas Git.

Verificação/comando	Resultado
rg para builders, master, config e /opt/conda	Nenhum hardcode remanescente nos scripts e helper efetivamente orquestrados. Exceção intencional: helper legado usado pelo Gold.
compile e execução dos imports via AST	Aprovados para 15 arquivos Silver/DQ; não houve execução dos corpos de processamento dos scripts.
Importação da DAG com <code>runpy.run_path</code>	Aprovada no ambiente Airflow.
<code>airflow dags list-import-errors</code>	Saída: No data found.
Asserções sobre a DAG importada	Quatro grupos, dez tasks, dependências, retries, connection e configurações conferidos.
<code>obter_periodos()</code>	Lista exatamente igual às 36 competências esperadas.
<code>processar_lote</code> com falha simulada	Todas as competências tentadas; exceção ao final.
Cinco <code>main()</code> DQ com relatório crítico simulado	Erro propagado e sessão encerrada.
Identidade dos helpers Gold/Silver	Gold mantém o helper original; Silver usa o novo.
<code>git diff -check</code>	Sem erros de whitespace; avisos Git de conversão LF/CRLF.
Revisão integral do diff e do arquivo novo	Sem mudanças funcionais ou alterações fora do escopo de implementação.

Houve ajustes no código temporário de verificação: a versão instalada de `DagBag` não aceita `include_examples`, e o operador expõe a connection em `_conn_id`, não em `conn_id`. Após adequar o verificador à API instalada, a execução final passou. Esses ajustes não exigiram mudanças na implementação da DAG.

O acesso inicial ao Docker foi bloqueado pelo sandbox. As verificações necessárias foram repetidas com autorização de execução, sem alterar permissões do filesystem.

7 Auditoria Gold: pendências preservadas

Nenhum arquivo Gold foi modificado. A lista abaixo registra as configurações encontradas no estado inspecionado:

Arquivo/local	Configuração	Classificação
src/gold/municipio_estacao.py, linhas 39–40	Master local[2]; UI false	Runtime
src/gold/fato_internacao.py, linha 47	spark.sql.autoBroadcastJoinThreshold = -1	Tuning
src/gold/fato_internacao_meteorologia.py, linhas 86–93	Shuffle 40; UI false	Tuning/runtime
DQs Gold via src/quality/gold/common.py	Reexportação do helper Silver original	Dependência compartilhada
src/quality/silver/common.py, linhas 7–9	Master local[2]; UI false; shuffle 4	Runtime/tuning herdados pelo Gold

O comentário em `fato_internacao.py` explica que o broadcast automático está desativado para evitar pressão sobre a memória do driver ao materializar dimensões. Esse ajuste foi apenas documentado, sem modificar joins ou planejamento.

Os três scripts de transformação Gold mantêm `appName` e log ERROR; os DQs recebem nomes de aplicação próprios e herdam o nível de log do helper. Não foram encontrados nesses arquivos hardcodes de memória, executor cores ou caminhos Python `/opt/conda`.

8 Estatísticas e revisão do diff

O comando `git diff --stat`, antes da criação deste relatório, apresentou:

```
dags/tcc_pipeline.py          | 35
src/quality/silver/dq_cid10.py |  2
src/quality/silver/dq_ibge.py  |  2
src/quality/silver/dq_inmet.py |  2
src/quality/silver/dq_inmet_diario.py |  2
src/quality/silver/dq_sih.py   |  2
src/silver/cid10.py            |  2
src/silver/ibge.py             |  2
src/silver/inmet.py            | 15
src/silver/inmet_diario.py     |  2
src/silver/sih.py              |  3
11 files changed, 33 insertions(+), 36 deletions(-)
```

Essas estatísticas excluem o novo `spark_session.py`, ainda não rastreado pelo Git. A implementação total envolve 11 arquivos existentes modificados e um arquivo novo. O Markdown e o PDF deste relatório são entregáveis documentais adicionais, produzidos por solicitação posterior.

O diff remove configurações dos builders, acrescenta configuração por task, troca cinco imports e ajusta uma docstring. O arquivo de lote SIH e o helper legado não mudaram. A revisão não identificou alteração de algoritmos, paths, schemas, valores esperados ou políticas de escrita.

9 Limitações e próximos passos

Os testes aprovados demonstram sintaxe/imports válidos, coerência da DAG e propagação de falhas simuladas. Não demonstram equivalência materializada de datasets, desempenho, consumo de memória nem execução de um job completo por `SparkSubmitOperator`.

Execuções manuais dos scripts refatorados agora precisam receber master e tuning pela ferramenta de submissão quando necessário; os scripts deixaram de impor esses valores. O container Spark de desenvolvimento não foi alterado.

Em uma execução operacional futura, será possível verificar o comando `spark-submit` efetivo, o interpretador usado pelos workers e os resultados dos gates com dados reais. A migração do Gold deverá tratar explicitamente o helper compartilhado e preservar os ajustes existentes de shuffle e broadcast. Essas atividades não foram executadas nesta tarefa.

10 Base documental

O relatório se baseia no pedido original, nos arquivos do repositório, no diff Git e nas saídas das verificações executadas nesta sessão. Não utiliza pesquisa externa. As versões declaradas foram conferidas nos arquivos do ambiente; as evidências de testes referem-se ao worker disponível no momento da execução.